Quick Sort

Data Structures Using C – Sorting Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recall: Merge Sort

Merge Sort follows:

$$\boxed{\text{Divide} \rightarrow \text{Conquer} \rightarrow \text{Combine}}$$

- Divides an array into two halves.
- Sorts the halves recursively.
- Merges the sorted halves.
- Time complexity: $\Theta(n \log n)$.
- Standard array version requires $O(n)$ auxiliary space.

Why Study Quick Sort?

Quick Sort is another divide-and-conquer algorithm.
However, unlike Merge Sort:

- Quick Sort partitions the array around a pivot.
- The pivot reaches its final position.
- The two partitions are recursively sorted.
- No separate merging operation is required.

Main Idea

Select Pivot $\rightarrow$ Partition $\rightarrow$ Recursively Sort

What is Quick Sort?

Quick Sort is a comparison-based sorting algorithm developed by C. A. R. Hoare.
It is also called:

Partition Exchange Sort

- It follows divide-and-conquer.
- It selects a pivot element.
- It partitions the array around the pivot.
- It recursively sorts the partitions.

Quick Sort: Basic Principle

Suppose:

$$A = [25,\ 10,\ 40,\ 15,\ 30]$$

Choose:

$$PIVOT = 25$$

After partitioning, one possible arrangement is:

$$[10,\ 15] \quad [25] \quad [40,\ 30]$$

Partition Property

Every element to the left of the pivot is less than or equal to it.
Every element to the right is greater than or equal to it.

What is a Pivot?

A pivot is the element used to divide an array into two partitions.
Common pivot selection methods:

1. First element
2. Last element
3. Middle element
4. Random element
5. Median-of-three

For This Lecture

We use the first element as the pivot, following the supplied textbook algorithm.

What is Partitioning?

Partitioning rearranges the elements so that:

$$A[left] \leq PIVOT$$

for elements in the left partition, and:

$$A[right] \geq PIVOT$$

for elements in the right partition.

After partitioning:

$$\underbrace{\text{Smaller Values}}_{\text{Left}} \quad \boxed{\text{Pivot}} \quad \underbrace{\text{Larger Values}}_{\text{Right}}$$

The pivot is in its final sorted position.

Quick Sort: Divide and Conquer

Divide	Conquer	Combine
Partition the array around a pivot.	Recursively sort the left and right partitions.	No merging is needed; the partitions are already positioned around the pivot.

$$QUICK_SORT(A, low, high)$$

Textbook Partition Technique

The textbook uses three important variables:

Variable	Meaning
LOC	Current location of pivot
LEFT	Left scanning position
RIGHT	Right scanning position

Initially:

$$LOC = LEFT = BEG$$

$$RIGHT = END$$

Partitioning: Right-to-Left Scan

Initially, the pivot is the first element.
Start scanning from the right.

While:

$$A[LOC] \leq A[RIGHT]$$

move:

$$RIGHT = RIGHT - 1$$

If:

$$A[LOC] > A[RIGHT]$$

swap the two values and set:

$$LOC = RIGHT$$

Partitioning: Left-to-Right Scan

After swapping with an element on the right, start scanning from the left.

While:

$$A[LOC] \geq A[LEFT]$$

move:

$$LEFT = LEFT + 1$$

If:

$$A[LOC] < A[LEFT]$$

swap the two values and set:

$$LOC = LEFT$$

When Does Partitioning Stop?

The scanning process continues alternately from right and left.

Partitioning stops when:

$$LOC = LEFT$$

or:

$$LOC = RIGHT$$

at the corresponding scan boundary.

Result

The pivot reaches its final position.
The partition function returns:

$$LOC$$

Quick Sort: Textbook Example

Consider:

$$A = [27,\ 1,\ 36,\ 18,\ 25,\ 45]$$



Choose the first element:

$$PIVOT = 27$$

Initially:

$$LOC = 0, \quad LEFT = 0, \quad RIGHT = 5$$

Partition: Initial Positions

Array:

$$[27,\ 1,\ 36,\ 18,\ 25,\ 45]$$

27	1	36	18	25	45
0	1	2	3	4	5

$$LOC = 0$$

$$LEFT = 0$$

$$RIGHT = 5$$

Partition: Compare Pivot with 45

Pivot:

$$PIVOT = 27$$

Rightmost value:

$$A[5] = 45$$

Since:

$$27 < 45$$

no swap is required.

Move the right pointer:

$$RIGHT = 4$$

Partition: Compare Pivot with 25

Current array:

$$[27,\ 1,\ 36,\ 18,\ 25,\ 45]$$

Right pointer:

$$RIGHT = 4$$

Compare:

$$27 > 25$$

Swap the pivot with 25.

Updated array:

$$\boxed{[25,\ 1,\ 36,\ 18,\ 27,\ 45]}$$

New pivot location:

$$LOC = 4$$

Partition: Scan from Left

Current array:

$$[25,\ 1,\ 36,\ 18,\ 27,\ 45]$$

Pivot remains:

$$PIVOT = 27$$

Current location:

$$LOC = 4$$

Compare from the left:

$$25 < 27$$

Move LEFT to index 1.

$$1 < 27$$

Move LEFT to index 2.

Partition: Compare 36 with Pivot

Current array:

$$[25,\ 1,\ 36,\ 18,\ 27,\ 45]$$

At:

$$LEFT = 2$$

Compare:

$$36 > 27$$

Swap 36 and 27.

Updated array:

$$\boxed{[25,\ 1,\ 27,\ 18,\ 36,\ 45]}$$

New pivot location:

$$LOC = 2$$

Partition: Scan from Right Again

Current array:

$$[25,\ 1,\ 27,\ 18,\ 36,\ 45]$$

Pivot location:

$$LOC = 2$$

Right pointer:

$$RIGHT = 4$$

Compare:

$$27 < 36$$

Move:

$$RIGHT = 3$$

Partition: Compare Pivot with 18

Current array:

$$[25,\ 1,\ 27,\ 18,\ 36,\ 45]$$

Compare:

$$27 > 18$$

Swap 27 and 18.

Updated array:

$$\boxed{[25,\ 1,\ 18,\ 27,\ 36,\ 45]}$$

New pivot location:

$$LOC = 3$$

Partition: Final Left Scan

Current array:

$$[25,\ 1,\ 18,\ 27,\ 36,\ 45]$$

Pivot:

$$PIVOT = 27$$

Continue scanning from:

$$LEFT = 2$$

Since:

$$18 < 27$$

move LEFT to index 3.
Now:

$$LEFT = LOC = 3$$

Partition Result

After the first partition:

$$[25,\ 1,\ 18,\ 27,\ 36,\ 45]$$

$$\underbrace{[25,\ 1,\ 18]}_{\text{Left Partition}} \quad \boxed{27} \quad \underbrace{[36,\ 45]}_{\text{Right Partition}}$$

The pivot 27 is now in its final sorted position.

$$LOC = 3$$

Quick Sort: Recursive Calls

After partitioning:

$$[25,\ 1,\ 18] \quad [27] \quad [36,\ 45]$$

Recursively sort:

$$QUICK_SORT(A, 0, 2)$$

and:

$$QUICK_SORT(A, 4, 5)$$

The pivot at index 3 is excluded from both recursive calls.

Sorting the Left Partition

Left partition:

$$[25,\ 1,\ 18]$$

Choose:

$$PIVOT = 25$$

Scan from right:

$$25 > 18$$

Swap:

$$[18,\ 1,\ 25]$$

Pivot 25 reaches its final position within the left partition.

Sorting the Remaining Left Partition

Remaining partition:

$$[18,\ 1]$$

Choose:

$$PIVOT = 18$$

Compare:

$$18 > 1$$

Swap:

$$[1,\ 18]$$

The left portion is now sorted:

$$\boxed{[1,\ 18,\ 25]}$$

Sorting the Right Partition

Right partition:

$$[36,\ 45]$$

Choose:

$$PIVOT = 36$$

Compare:

$$36 < 45$$

No swap is required.

The right partition is already sorted:

$$\boxed{[36,\ 45]}$$

Quick Sort: Final Result

Initial array:

$$[27,\ 1,\ 36,\ 18,\ 25,\ 45]$$

After sorting:

$$\boxed{[1,\ 18,\ 25,\ 27,\ 36,\ 45]}$$

Important

Quick Sort does not require a separate merge operation after the recursive calls.

Quick Sort: Recursive Structure

Initial array:

[27, 1, 36, 18, 25, 45]

↓

[25, 1, 18] [27] [36, 45]

↓

[18, 1] [25] [27] [36, 45]

↓

Partition Algorithm: Initialization

PARTITION(A, BEG, END)

1. Set:

$$LEFT = BEG$$

$$RIGHT = END$$

$$LOC = BEG$$

2. Set:

$$FLAG = 0$$

3. Repeat while $FLAG = 0$.

Partition Algorithm: Right Scan

Right-to-Left Scanning

1. While:
$$LOC \neq RIGHT \quad \text{and} \quad A[LOC] \leq A[RIGHT]$$
decrement RIGHT.
2. If $LOC = RIGHT$, set $FLAG = 1$.
3. Otherwise:
   - Swap $A[LOC]$ and $A[RIGHT]$.
   - Set $LOC = RIGHT$.

Partition Algorithm: Left Scan

Left-to-Right Scanning

If $FLAG = 0$:

1. While:
$$LOC \neq LEFT \quad \text{and} \quad A[LOC] \geq A[LEFT]$$
increment LEFT.
2. If $LOC = LEFT$, set $FLAG = 1$.
3. Otherwise:
   - Swap $A[LOC]$ and $A[LEFT]$.
   - Set $LOC = LEFT$.

Return LOC when partitioning terminates.

Quick Sort: Recursive Algorithm

QUICK_SORT(A, BEG, END)

1. If $BEG < END$:
   1. Call:
      $$LOC = PARTITION(A, BEG, END)$$
   2. Call:
      $$QUICK_SORT(A, BEG, LOC - 1)$$
   3. Call:
      $$QUICK_SORT(A, LOC + 1, END)$$
2. Return.

Partition Function in C: Part 1

```
1  int partition(int a[], int beg, int end)
2  {
3      int loc = beg;
4      int left = beg;
5      int right = end;
6      int flag = 0;
7
8      while (flag == 0)
9      {
10         while (loc != right &&
11                a[loc] <= a[right])
12         {
13             right--;
14         }
15
16         if (loc == right)
17             flag = 1;
18         else
19         {
20             int temp = a[loc];
21             a[loc] = a[right];
22             a[right] = temp;
23             loc = right;
24         }
```

Partition Function in C: Part 2

```
1        if (flag == 0)
2        {
3            while (loc != left &&
4                   a[loc] >= a[left])
5            {
6                left++;
7            }
8
9            if (loc == left)
10               flag = 1;
11           else
12           {
13               int temp = a[loc];
14               a[loc] = a[left];
15               a[left] = temp;
16               loc = left;
17           }
18       }
19   }
20
21   return loc;
22 }
```

Quick Sort: Recursive C Function

```
1  void quickSort(int a[], int beg, int end)
2  {
3      if (beg < end)
4      {
5          int loc = partition(a, beg, end);
6
7          quickSort(a, beg, loc - 1);
8
9          quickSort(a, loc + 1, end);
10     }
11 }
```

Base Case

Recursion terminates when:

$$BEG \geq END$$

Complete C Program: Part 1

```
#include <stdio.h>

int partition(int a[], int beg, int end)
{
    int loc = beg, left = beg, right = end;
    int flag = 0;

    while (!flag)
    {
        while (loc != right &&
               a[loc] <= a[right])
            right--;

        if (loc == right)
            flag = 1;
        else
        {
            int t = a[loc];
            a[loc] = a[right];
            a[right] = t;
            loc = right;
        }

        if (!flag)
        {
            while (loc != left &&
                   a[loc] >= a[left])
                left++;
```

Complete C Program: Part 2

```
            if (loc == left)
                flag = 1;
            else
            {
                int t = a[loc];
                a[loc] = a[left];
                a[left] = t;
                loc = left;
            }
        }
    }
    return loc;
}

void quickSort(int a[], int beg, int end)
{
    if (beg < end)
    {
        int loc = partition(a, beg, end);
        quickSort(a, beg, loc - 1);
        quickSort(a, loc + 1, end);
    }
}
```

Complete C Program: Part 3

```c
int main(void)
{
    int a[] = {27, 1, 36, 18, 25, 45};
    int n = sizeof(a) / sizeof(a[0]);

    quickSort(a, 0, n - 1);

    printf("Sorted array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

Expected Output

```
Sorted array:
1 18 25 27 36 45
```

Quick Sort: Best Case

The best case occurs when the pivot divides the array into approximately equal halves.

Recurrence:

$$T(n) = 2T(n/2) + \Theta(n)$$

Using the Master Theorem:

$$\boxed{T(n) = \Theta(n \log n)}$$

Recursion depth:

$$O(\log n)$$

Quick Sort: Worst Case

The worst case occurs when partitions are extremely unbalanced.
For example:

$$[1,\ 2,\ 3,\ 4,\ 5]$$

If the first element is always chosen as pivot, one partition is empty.

Recurrence:

$$T(n) = T(n-1) + \Theta(n)$$

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

Quick Sort: Average Case

With reasonably balanced partitions on average:

$$T(n) = \Theta(n \log n)$$

Case	Time Complexity
Best	$\Theta(n \log n)$
Average	$\Theta(n \log n)$
Worst	$\Theta(n^2)$

Important

Random pivot selection improves expected performance but does not eliminate the possibility of quadratic worst-case time.

Quick Sort: Space Complexity

Quick Sort partitions the array in place.
However, recursion requires stack space.

Case	Recursion Stack
Balanced partitions	$O(\log n)$
Highly unbalanced partitions	$O(n)$

Observation

Quick Sort uses constant additional storage for partitioning, but its recursive implementation is not strictly $O(1)$ in total auxiliary space.

Is Quick Sort Stable?

Consider:

$$[(4, A),\ (2, B),\ (4, C),\ (1, D)]$$

A standard in-place partition may change the relative order of equal keys.

Observation

Standard in-place Quick Sort is generally **not stable**.

Stable variants exist, but they may require additional memory or different partitioning techniques.

Quick Sort vs Merge Sort

Property	Quick Sort	Merge Sort
Strategy	Partition	Divide and merge
Best time	$O(n \log n)$	$O(n \log n)$
Average time	$O(n \log n)$	$O(n \log n)$
Worst time	$O(n^2)$	$O(n \log n)$
Auxiliary space	Stack dependent	$O(n)$
Stable	Usually no	Yes
Standard array implementation	In-place partition	Extra buffer

Advantages and Limitations

Advantages

- Efficient average-case time.
- Good practical performance.
- In-place partitioning.
- Suitable for array sorting.
- Divide-and-conquer approach.

Limitations

- Quadratic worst case.
- Usually unstable.
- Recursive stack overhead.
- Performance depends on pivot selection.
- Partition logic is more complex.

Classroom Activity 1

Apply the first-element pivot Quick Sort technique to:

$$[40,\ 20,\ 60,\ 10,\ 50,\ 30]$$

Tasks:

1. Identify the pivot.
2. Show right-to-left scanning.
3. Show left-to-right scanning.
4. Show each swap.
5. Identify the final pivot position.
6. Identify the two resulting partitions.

Classroom Activity 1: Solution

Initial:

$$[40,\ 20,\ 60,\ 10,\ 50,\ 30]$$

Pivot:

$$40$$

Right scan finds 30:

$$[30,\ 20,\ 60,\ 10,\ 50,\ 40]$$

Left scan finds 60:

$$[30,\ 20,\ 40,\ 10,\ 50,\ 60]$$

Right scan finds 10:

$$[30,\ 20,\ 10,\ 40,\ 50,\ 60]$$

Pivot position:

$$LOC = 3$$

Classroom Activity 2

Consider the already sorted array:

$$[10,\ 20,\ 30,\ 40,\ 50]$$

Use the first element as pivot.

Answer:

1. Which element is selected as the first pivot?
2. What are the resulting partition sizes?
3. Will the partitions be balanced?
4. What is the recurrence relation?
5. What is the resulting time complexity?

Classroom Activity 2: Solution

- First pivot: 10
- Left partition: empty
- Right partition: four elements
- Partitions are highly unbalanced

Recurrence:

$$T(n) = T(n-1) + \Theta(n)$$

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

MCQ 1

Quick Sort uses which design strategy?

1. Dynamic programming
2. Divide and conquer
3. Backtracking
4. Greedy algorithm

MCQ 1

Quick Sort uses which design strategy?

1. Dynamic programming
2. Divide and conquer
3. Backtracking
4. Greedy algorithm

Answer

Option 2: Divide and conquer.

MCQ 2

What is the average-case time complexity of Quick Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

MCQ 2

What is the average-case time complexity of Quick Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

Answer

Option 3: $O(n \log n)$.

MCQ 3

After partitioning, which statement is correct?

1. The entire array is sorted.
2. The pivot is in its final position.
3. All elements are equal.
4. The array is divided into equal-sized halves.

MCQ 3

After partitioning, which statement is correct?

1. The entire array is sorted.
2. The pivot is in its final position.
3. All elements are equal.
4. The array is divided into equal-sized halves.

Answer

Option 2: The pivot is in its final position.

MCQ 4

What is the worst-case time complexity of Quick Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

MCQ 4

What is the worst-case time complexity of Quick Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

Answer

Option 4: $O(n^2)$.

Common Programming Mistakes

1. Incorrect partition boundaries.
2. Missing recursion base case.
3. Incorrect pivot location updates.
4. Moving LEFT or RIGHT beyond valid bounds.
5. Recursively including the pivot again.
6. Incorrect handling of duplicate values.
7. Assuming random pivot selection guarantees $O(n \log n)$ worst-case time.

Practice Questions

1. Explain Quick Sort with an example.
2. Define pivot and partitioning.
3. Explain the first-element pivot technique.
4. Trace Quick Sort on:
$$[27,\ 1,\ 36,\ 18,\ 25,\ 45]$$
5. Write the partition algorithm.
6. Implement Quick Sort in C.
7. Derive best- and worst-case recurrences.
8. Compare Quick Sort and Merge Sort.

Lecture Summary

Key Concepts

- Quick Sort follows divide and conquer.
- A pivot is selected for partitioning.
- The pivot reaches its final position.
- Left and right partitions are recursively sorted.
- Average time: $\Theta(n \log n)$.
- Worst-case time: $\Theta(n^2)$.
- Standard in-place Quick Sort is generally unstable.

References

1. Reema Thareja, *Data Structures Using C*, chapter on Searching and Sorting.
2. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Pivot selection
- Partitioning technique
- Pointer movements
- Recursive Quick Sort
- C implementation
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Pivot $\rightarrow$ Partition $\rightarrow$ Recursion